Reviewer Pack — Minimal Galois Group Complexity of Boolean Functions and Cir…

Entry 007a07483b36 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 17:11:37 UTC

Minimal Galois Group Complexity of Boolean Functions and Circuit Entanglement

Entry ID: `007a07483b36` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 17:11:37 UTC

1. Conjecture

Field A (mathematical branch): Galois Representation Theory **Field B** (complexity object): Boolean Circuits (Circuit Entanglement Complexity)

Statement:

> For every d-regular boolean function f, the minimal degree of a Galois representation that computes f is bounded by the maximum entanglement of any circuit computing f, such that deg(G_f) = O(Ent(f)) and Ent(f) ≤ deg(G_f)^2 for all circuits C_f with entanglement Ent(C_f).

Rationale (proposer's reasoning):

> Galois representations provide a way to encode functions algebraically, which may reveal hidden structures in boolean functions. The conjecture suggests that the complexity of these representations could be related to circuit entanglement, a measure of the difficulty of decomposing computations into independent sub-computations.

Taxonomy category: `Galois-Representation-Boolean-Circuit` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cc291c4495c3f2b6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across at least 90% of the 30 seeds, the degree of the Galois group (deg(G_f)) for each d-regular boolean function f is within O(Ent(f)), and the entanglement Ent(f) is less than or equal to the square of deg(G_f)^2 for all circuits C_f. The conjecture is falsified if this condition fails for any seed.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Galois representation theory AND Boolean circuit entanglement` `- degree of Galois representation AND boolean function complexity` `- entanglement in Boolean circuits AND degree of Galois group`

Top relevant hits considered: - [http://arxiv.org/abs/2111.08018v2] Entanglement dynamics in hybrid quantum circuits - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/1101.4339v3] Galois theory of quadratic rational functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/2004.12063v2] Hardness of Random Optimization Problems for Boolean Circuits, Low-Degree Polynomials, and Langevin Dynamics - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_boolean_function(d, n):
        if d <= 0 or n <= 0:
            return None
        function = [random.choice([0, 1]) for _ in range(n)]
        while True:
            valid = True
            for i in range(n):
                count = sum(1 for j in range(n) if (i != j and function[i] == function[j]))
                if count != d:
                    valid = False
                    break
            if valid:
                return function
            function = [random.choice([0, 1]) for _ in range(n)]

    def calculate_galois_group_degree(function):
        n = len(function)
```

```
        if n <= 1:
            return 0
        degree = 1
        while True:
            new_function = [function[i] ^ function[(i + degree) % n] for i in range(n)]
            if new_function == function:
                break
            degree += 1
        return degree

    def calculate_circuit_entanglement(function):
        n = len(function)
        entanglement = 0
        for i in range(n):
            count = sum(1 for j in range(n) if (i != j and function[i] == function[j]))
            entanglement = max(entanglement, count)
        return entanglement

    instances_tested = 0
    n_max = 5
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            d = random.randint(1, min(n - 1, 5))
            function = generate_d_regular_boolean_function(d, n)
            if function is None:
                continue
            instances_tested += 1
            n_max = max(n_max, n)
            deg_G_f = calculate_galois_group_degree(function)
            Ent_f = calculate_circuit_entanglement(function)
            if deg_G_f > Ent_f * Ent_f or Ent_f > deg_G_f:
                conjecture_holds = False
                counterexample = f"n={n}, d={d}, deg(G_f)={deg_G_f}, Ent(f)={Ent_f}"
                break

    return {
        "metric_name": "Galois Group Degree",
        "metric_value": deg_G_f,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```python
mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={supp
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={supp
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, making it impossible to verify the conjecture's conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14231
2	preregistration	ollama_remote	glm4:latest	0	0	11614
3	novelty	ollama_remote	glm4:latest	0	0	8355
4	novelty	ollama_remote	glm4:latest	0	0	10052
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16573
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7696
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10851
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10462
9	judge	ollama_remote	glm4:latest	0	0	11468

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 101303 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/007a07483b36.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/007a07483b36.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/007a07483b36.tar.gz` (if generated)